Sorting and Searching Project

Course: `CSCI 236: Data Structures and Algorithms`
Semester: Fall 2025

Due: 10:00 AM, Wednesday, October 1

Overview

Sorting is a fundamental problem in computer science. It plays a key role in many everyday applications, such as indexing documents and organizing memory structures efficiently.

In this project, you will:

- Implement three sorting algorithms: **QuickSort**, **MergeSort**, and **InsertionSort**.
- Implement **Binary Search** to support character indexing.
- Sort words using a custom-defined alphabet from a configuration file.
- Measure algorithm performance and analyze behavior across varying input sizes.

Input Format

Your program will accept two filenames as command-line arguments:

```
python sorter_framework.py alphabet.txt wordlist.txt
```

- `alphabet.txt`: Contains one character per line, defining a custom sort order.
  Sample Input:

  ```
  c
  a
  b
  d
  e
  ```

- `wordlist.txt`: Contains one word per line. All characters must be present in the custom alphabet.
  Sample Input:

  ```
  cab
  bed
  bad
  ace
  bee
  cad
  dab
  ```

Note: Whitespace characters (e.g., space or tab) are valid and may appear in words or the alphabet.

Output

After each sorting algorithm completes, the terminal should display the following information for each sort:

- The name of the sorting algorithm
- The length of the alphabet list
- The total number of words sorted
- The time taken to sort the list (in milliseconds)
- The first 10 words of the sorted list

Times should be reported with at least millisecond precision.

Expected Output (Terminal)

```
--- InsertionSort ---
Alphabet count: 5
Word count: 7
Time: 0.08 ms
First 10 words: ['cab', 'cad', 'ace', 'bad', 'bed', 'bee', 'dab']

--- MergeSort ---
Alphabet count: 5
Word count: 7
Time: 0.06 ms
First 10 words: ['cab', 'cad', 'ace', 'bad', 'bed', 'bee', 'dab']

--- QuickSort ---
Alphabet count: 5
Word count: 7
Time: 0.04 ms
First 10 words: ['cab', 'cad', 'ace', 'bad', 'bed', 'bee', 'dab']
```

Explanation of Sort Order

The custom alphabet defines the character order:

`c < a < b < d < e`

Words are compared letter-by-letter using this order, which results in the sorted list:

`['cab', 'cad', 'ace', 'bad', 'bed', 'bee', 'dab']`

Required Files and Classes

You are provided with a partially completed project structure consisting of multiple Python files. Some files have already been implemented for you, while others (marked in red) require your implementation.

Files you must complete (implement):

binary_search.py
: Implement binary search to efficiently locate characters in the custom alphabet.

insertion_sorter.py
: Implement the Insertion Sort algorithm.

merge_sorter.py
: Implement the Merge Sort algorithm.

quick_sorter.py
: Implement the Quick Sort algorithm.

sorter_framework.py
: Implement the main execution logic: loading files, running sorts, collecting statistics, and printing output.

Files already provided (do not modify):

alphabet.py Defines the `Alphabet` class, which stores the custom character order and provides support functions for lookup.

alphabet_comparator.py
: Defines the `AlphabetComparator` class, which is used to compare two strings using the custom alphabet.

wordlist.py Contains the `WordList` class, which holds the list of words and provides cloning functionality.

sorter.py An abstract base class for all sorting algorithms. It includes functionality for tracking sort statistics and defines the `sort()` interface.

Important: You should not modify the provided files unless specifically instructed. Your task is to implement the missing functionality in the red-highlighted files according to the assignment requirements.

Additional Requirements

You must test your sorting algorithms on word lists of the following sizes:

$$n = 10, \quad 100, \quad 1{,}000, \quad 10{,}000, \quad 100{,}000, \quad 1{,}000{,}000$$

For each size, the following test files are provided:

- `<n>.alphabet.txt` – Custom alphabet (1 character per line)
- `<n>.wordlist.txt` – List of n words to be sorted
- `<n>.sortedlist.txt` – Expected sorted output (for verification)

Use these files to:

- Validate your output against `<n>.sortedlist.txt`
- Measure performance and comparison statistics
- Write a discussion analyzing the results and trends observed in your `analysis.txt`.

Note: You may skip `InsertionSort` for large inputs if it takes too long. If you do, include how long it ran before you stopped it, and explain this decision in your analysis report.

Example usage:

```
python sorter_framework.py 1000.alphabet.txt 1000.wordlist.txt
```

Performance Analysis Report

Create a plain text file named `analysis.txt` (maximum 500 words) that discusses:

- How the measured results compare to theoretical expectations (Big-O)
- How performance changes with different input sizes
- Which algorithm works best for small vs. large inputs
- Any unexpected or interesting behaviors observed

Submission Instructions

Submit a single zip file named:

```
Firstname_Lastname_HW2.zip
```

It should contain the following:

```
Firstname_Lastname_HW2/
 alphabet.py
 alphabet_comparator.py
 binary_search.py
 insertion_sorter.py
 merge_sorter.py
 quick_sorter.py
 sorter.py
 sorter_framework.py
 wordlist.py
 analysis.txt
```

Grading Rubric

Component	Points
Correct algorithm implementations	40
Comparator and binary search logic	20
Performance tracking and statistics	15
Analysis report	10
Code structure and clarity	10
Documentation and submission format	5
Total	**100**

Good luck, and happy sorting!